

Operating Systems and System Programming

Practical – 7

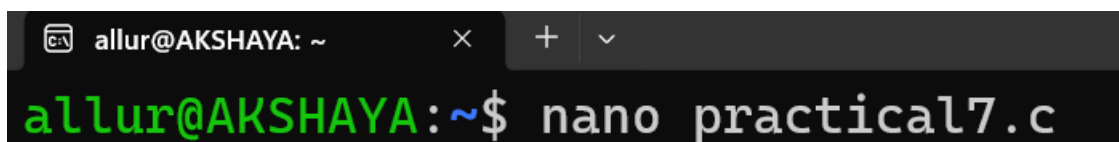
Aim

To write a C program to print the addresses of code, global, static, heap, and stack variables, analyze the Linux process address space, and study virtual memory mappings using /proc/<PID>/maps and memory analysis tools.

Algorithm

1. Start the program.
2. Declare global, static, and local variables.
3. Allocate memory dynamically using malloc().
4. Print the addresses of code, global, static, heap, and stack variables.
5. Display the process ID using getpid().
6. Keep the process running using sleep().
7. Open another terminal and examine /proc/<PID>/maps.
8. Identify the code, data, heap, stack, and shared library mappings.
9. Analyze memory addresses and permissions using pmap.
10. Free allocated memory and terminate the program.

Step-1:

A terminal window with a dark background. The title bar shows 'allur@AKSHAYA: ~' and window control buttons. The command prompt is 'allur@AKSHAYA: ~\$' and the command entered is 'nano practical7.c'.

```
allur@AKSHAYA: ~$ nano practical7.c
```

Step-2:

```
allur@AKSHAYA: ~
GNU nano 8.7.1 practical7.c *

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;

void display()
{
    printf("Code address:  %p\n", (void *)display);
}

int main()
{
    static int static_var = 20;
    int local_var = 30;
    int *heap_var = malloc(sizeof(int));

    if (heap_var == NULL)
    {
        perror("malloc");
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Exe
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Jus
```

```
allur@AKSHAYA: ~
GNU nano 8.7.1 practical7.c *

        perror("malloc");
        return 1;
    }

    *heap_var = 40;

    display();
    printf("Global address: %p\n", (void *)&global_var);
    printf("Static address: %p\n", (void *)&static_var);
    printf("Heap address:  %p\n", (void *)heap_var);
    printf("Stack address: %p\n", (void *)&local_var);

    printf("Process ID: %d\n", getpid());
    printf("Process is running...\n");
    fflush(stdout);

    sleep(60);

    free(heap_var);
    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execu
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justi
```

Step-3:

```
allur@AKSHAYA:~$ gcc practical7.c -o practical7
allur@AKSHAYA:~$ ./practical7
Code address: 0x5f249d672249
Global address: 0x5f249d675010
Static address: 0x5f249d675014
Heap address: 0x5f24b4302010
Stack address: 0x7ffe74f34e3c
Process ID: 524
Process is running...
```

While the program is running, open another terminal and run

```
allur@AKSHAYA:~$ cat /proc/524/maps
5f249d671000-5f249d672000 r--p 00000000 08:30 40888 /home/allur/practical7
5f249d672000-5f249d673000 r-xp 00001000 08:30 40888 /home/allur/practical7
5f249d673000-5f249d674000 r--p 00002000 08:30 40888 /home/allur/practical7
5f249d674000-5f249d675000 r--p 00002000 08:30 40888 /home/allur/practical7
5f249d675000-5f249d676000 rw-p 00003000 08:30 40888 /home/allur/practical7
5f24b4302000-5f24b4323000 rw-p 00000000 00:00 0 [heap]
7e2d06600000-7e2d06628000 r--p 00000000 08:30 13645 /usr/lib/x86_64-linux-gnu/libc.so.6
7e2d06628000-7e2d0667c000 r-xp 00028000 08:30 13645 /usr/lib/x86_64-linux-gnu/libc.so.6
7e2d0667c000-7e2d06680e000 r--p 001c0000 08:30 13645 /usr/lib/x86_64-linux-gnu/libc.so.6
7e2d06680e000-7e2d066812000 r--p 0020d000 08:30 13645 /usr/lib/x86_64-linux-gnu/libc.so.6
7e2d066812000-7e2d066814000 rw-p 00211000 08:30 13645 /usr/lib/x86_64-linux-gnu/libc.so.6
7e2d066814000-7e2d066821000 rw-p 00000000 00:00 0
7e2d066827000-7e2d06682a000 rw-p 00000000 00:00 0
7e2d066830000-7e2d066832000 rw-p 00000000 00:00 0
7e2d066832000-7e2d066836000 r--p 00000000 00:00 0 [vvar]
7e2d066836000-7e2d066838000 r--p 00000000 00:00 0 [vvar_vclock]
7e2d066838000-7e2d06683a000 r-xp 00000000 00:00 0 [vdso]
7e2d06683a000-7e2d06683b000 r--p 00000000 08:30 13642 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e2d06683b000-7e2d06686a000 r-xp 00001000 08:30 13642 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e2d06686a000-7e2d066875000 r--p 00030000 08:30 13642 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e2d066875000-7e2d066877000 r--p 0003b000 08:30 13642 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e2d066877000-7e2d066878000 rw-p 0003d000 08:30 13642 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7e2d066878000-7e2d066879000 rw-p 00000000 00:00 0
7ffe74f17000-7ffe74f38000 rw-p 00000000 00:00 0 [stack]
allur@AKSHAYA:~$ pmap 524
allur@AKSHAYA:~$
```

Result

The addresses of code, global, static, heap, and stack variables were successfully displayed. The Linux process address space and virtual memory mappings were studied using `/proc/<PID>/maps` and `pmap`, and the memory regions and their permissions were analyzed.